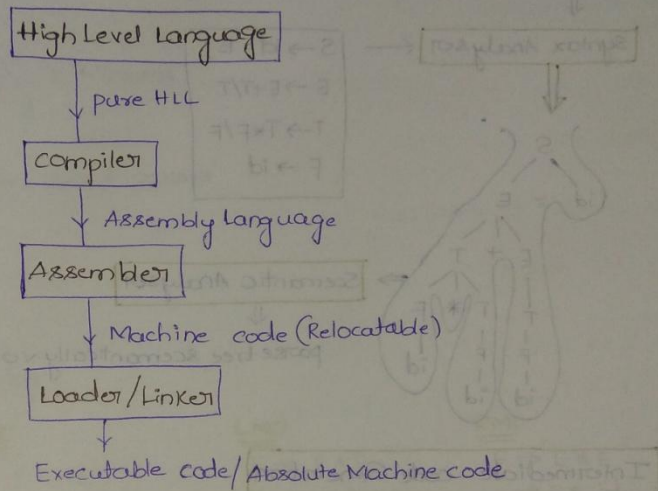


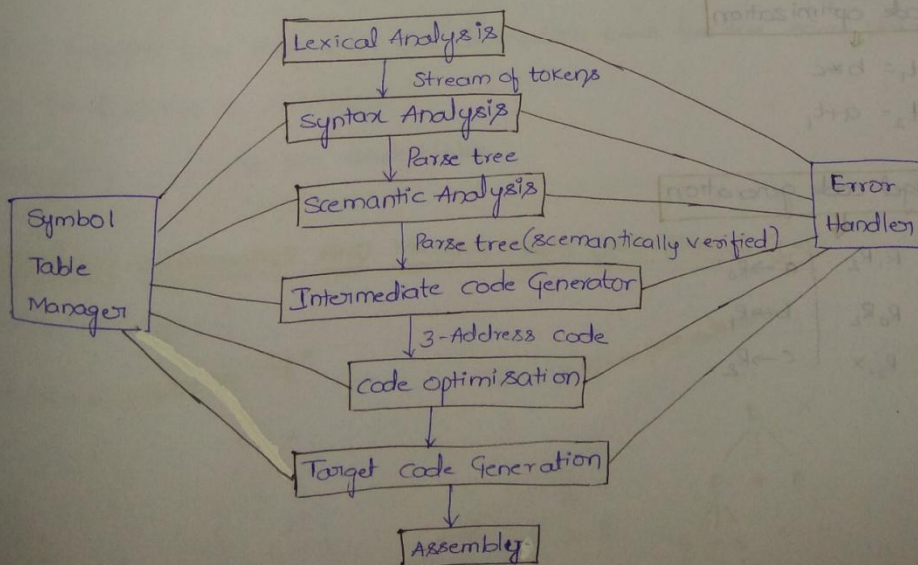
INTRODUCTION AND VARIOUS PHASES OF COMPILER (L-1)

⇒ The main aim of the compiler is to convert a High Level language to Low level language.



⇒ Removing `#include` by including a specific file is called "File Inclusion".

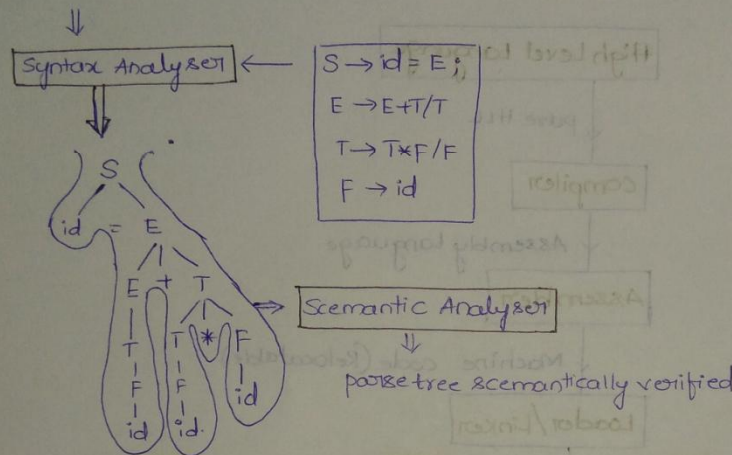
⇒ "Assembler" is dependent on the platform [Hardware + OS].



Example : INTRODUCTION AND VARIOUS PHASES OF COMPILER

1) $x = a + b * c$

↓
Lexical Analysis ⇒ The Lexical Analyzer identifies the "identifiers", "tokens" is using some Regular expressions called patterns $(id + d)^*$
 $id = id + id * id$



Intermediate code Generation

↓
 $t_1 = b * c$
 $t_2 = a + t_1$
 $x = t_2$

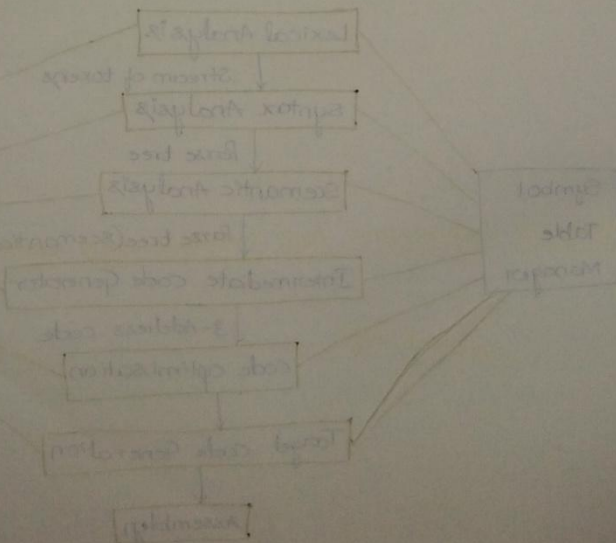
Code optimization

↓
 $t_1 = b * c$
 $t_2 = a + t_1$

Target code generation

↓

Mul R_1, R_2		$a \rightarrow R_0$
Add R_0, R_2		$b \rightarrow R_1$
Mov R_2, x		$c \rightarrow R_2$



②

INTRODUCTION TO LEXICAL ANALYSER (L-2)

③

⇒ This is the only phase that reads the input character by character

⇒ int max(x,y)

int x,y;

/* find max of x and y */

{

return (x > y ? x : y);

}

int = 1 token

max = 1 token

return = 1 token

95 Tokens are present

⇒ printf("%d Hai %d", x);

1 2 3 4 5 6 7 8

⇒ 8 Tokens

⇒ Syntax Analyser is also called parser

Grammar:

$E \rightarrow E + E / E * E / id$

⇒ id + id * id = Given string

LMD

$E \rightarrow E + E$

→ id + E * E

→ id + id * E

→ id + id * id

RMD

$E \rightarrow E + E$

→ E + E * E

→ E + E * id

→ E + id * id

→ id + id * id

LMD

$E \rightarrow E * E$

→ E + E * E

→ id + E * E

→ id + id * E

→ id + id * id

RMD

$E \rightarrow E * E$

→ E * id

→ E + E * id

→ E + id * id

→ id + id * id

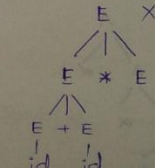
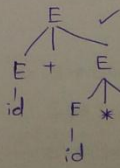
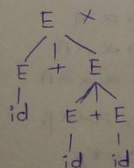
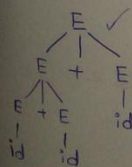
⇒ If a Grammar has more than one derivation trees for the same string then the Grammar is Ambiguous

⇒ Ambiguity problems are undecidable

AMBIGUOUS GRAMMARS AND MAKING THEM UNAMBIGUOUS (L-3)

$E \rightarrow E + E / E * E / id$

id + id + id ⇒ Associativity x



Leftmost plus should be evaluated first

Highest precedence one should be evaluated first (* should be evaluated first)

$$\Rightarrow \text{Pr}^*(A \rightarrow \text{Pr}^*)$$

$$\begin{matrix} A \rightarrow \text{Pr}^* \\ A' \rightarrow \alpha A' / \epsilon \end{matrix}$$

$$\Leftrightarrow \begin{matrix} A \rightarrow \alpha A' / \beta \\ A' \rightarrow \epsilon / \alpha A' \end{matrix}$$

PARSER

If the grammar is of the form
 $A \rightarrow \alpha A / \beta$ then eliminate Left Recursion
 by $\begin{matrix} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' / \epsilon \end{matrix}$

$$\begin{matrix} E \rightarrow E + I / T \\ A \rightarrow \alpha / \beta \end{matrix}$$

$$\begin{matrix} E \rightarrow TE' \\ E' \rightarrow \epsilon / +TE' \end{matrix}$$

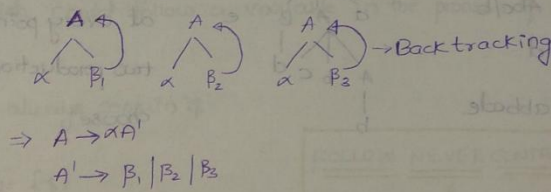
= Left Recursion Eliminated

$$\begin{matrix} A \rightarrow B_1 A' / B_2 A' / B_3 A' \\ A' \rightarrow \alpha_1 A' / \alpha_2 A' / \alpha_3 A' \end{matrix} \left\} \text{ is the eliminated LR of the Grammar } A \rightarrow \alpha_1 / \alpha_2 / \alpha_3 / \dots / \beta_1 / \beta_2 / \beta_3 \dots$$

3) Grammars can be classified into various categories

G_1

Ambiguous	Unambiguous
Left Recursive	Right Recursive
Deterministic	Non-Deterministic ($A \rightarrow \alpha \beta / \alpha \beta_2 / \alpha \beta_3$)



$$\begin{matrix} \textcircled{1} S \rightarrow iEtS / iEtSeS/a \\ E \rightarrow b \end{matrix} \quad \left| \quad \begin{matrix} S \rightarrow iEtSS'/a \\ S' \rightarrow eS/\epsilon \\ E \rightarrow b \end{matrix}$$

$\Rightarrow$ The elimination of Non-determinism doesn't guarantee the elimination of Ambiguity.

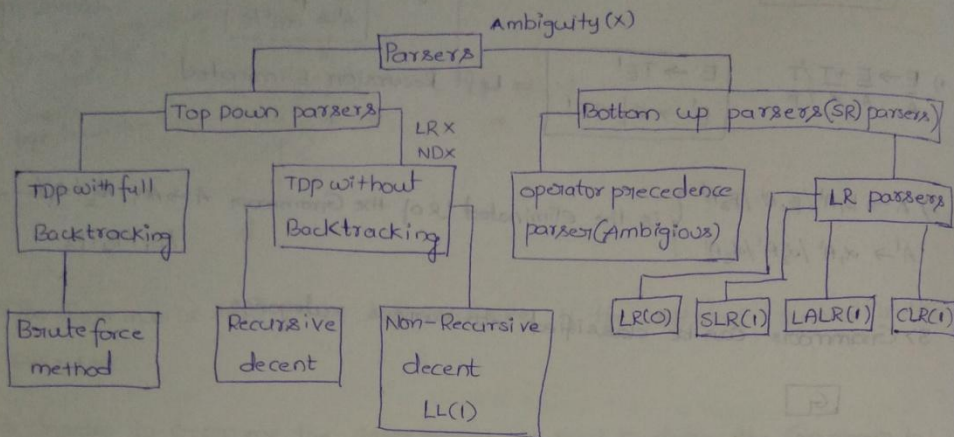
$$\textcircled{2} S \rightarrow \underline{a}ssbs / \underline{a}sasb / \underline{a}bb/b \Rightarrow \begin{matrix} S \rightarrow \underline{a}S' / b \\ S' \rightarrow \underline{s}sbs / \underline{s}asb / \underline{b}b \\ S' \rightarrow SS'' \\ S'' \rightarrow sb / asb \end{matrix}$$

$$\textcircled{3} S \rightarrow bSSaas / bSSaasb / bsb/a \Rightarrow \begin{matrix} S \rightarrow bSS' / a \\ S' \rightarrow \underline{s}aas / \underline{s}asb / b \\ S' \rightarrow SsS'' / b \\ S'' \rightarrow as / sb / \end{matrix}$$

PARSERS

(C)

⇒ parsers are nothing but Syntax Analyzers. (L-5)



FIRST():

$S \rightarrow aABc$

$A \rightarrow b$

$B \rightarrow c$

$C \rightarrow d$

$D \rightarrow e$

$S \rightarrow ABCD$

$A \rightarrow b$

$B \rightarrow c$

$C \rightarrow d$

$D \rightarrow e$

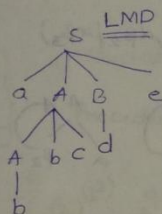
⇒

$S \rightarrow aABe$

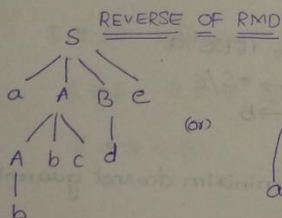
$A \rightarrow Abc/b$

$B \rightarrow d$

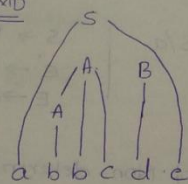
$w = abbade$



The main thing that we must decide is at every point when we have more than two productions to be chosen, "which one to choose"



(or)



⇒ The main decision that should be made is if I see a terminal when to Reduce

FOLLOW():

⇒ what is

derivation

⇒ follow a

$S \rightarrow ABCD$

$A \rightarrow b/e$

$B \rightarrow c/e$

$C \rightarrow d$

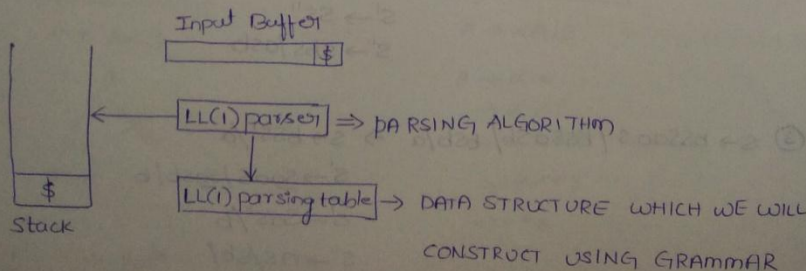
$D \rightarrow e$

⇒ Now, if n

variable

LL(1) PARSER / NON RECURSIVE DECENT PARSER

Left to Right, Left most Derivation, (1) = No. of look aheads



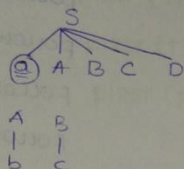
⑥

Now, Before constructing the LL(1) parsing table we should know two functions they are FIRST AND FOLLOW

FIRST():

$S \rightarrow aABCD$

$A \rightarrow b$
 $B \rightarrow c$
 $C \rightarrow d$
 $D \rightarrow e$

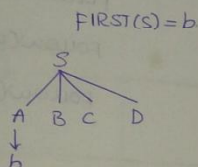


$FIRST(S) = a$
 $FIRST(A) = b$
 $FIRST(B) = c$

$FIRST(C) = d$
 $FIRST(D) = e$

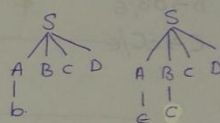
$S \rightarrow ABCD$

$A \rightarrow b$
 $B \rightarrow c$
 $C \rightarrow d$
 $D \rightarrow e$



$S \rightarrow ABCD$
 $A \rightarrow b/e$
 $B \rightarrow c \text{ (small 'c')}$
 $C \rightarrow d$
 $D \rightarrow e$

$FIRST(S) = \{b, c\}$



FOLLOW():

→ what is the terminal which could follow a variable in the process of derivation.

→ follow of start symbol always contain '\$'

$S \rightarrow ABCD$

$FOLLOW(S) = \{\$ \}$

$A \rightarrow b/e$

$FOLLOW(A) = FIRST(B, C, D) = \{c, d, e\}$

$B \rightarrow c/e$

$FOLLOW(B) = FIRST(D) = \{d\}$

$C \rightarrow d$

$FOLLOW(C) = FIRST(D) = \{e\}$

$D \rightarrow e$

FOLLOW NEVER CONTAIN
"EPSILON"

→ Now, If nothing is following a variable i.e. If nothing is at the RHS of a variable then the follow of that variable is the follow of LHS

$\therefore FOLLOW(D) = FOLLOW(S) = \{\$ \}$

EXAMPLES ON HOW TO FIND FIRST AND FOLLOW IN LL(1) PARSER (L-6)

CONSTRUCT

1) $S \rightarrow ABCDE$

$A \rightarrow a/\epsilon$

$B \rightarrow b/\epsilon$

$C \rightarrow c$

$D \rightarrow d/\epsilon$

$E \rightarrow e/\epsilon$

$FIRST(S) = \{a, b, c\}$

$FIRST(A) = \{a, \epsilon\}$

$FIRST(B) = \{b, \epsilon\}$

$FIRST(C) = \{c\}$

$FIRST(D) = \{d, \epsilon\}$

$FIRST(E) = \{e, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{b, c\}$

$FOLLOW(B) = \{c\}$

$FOLLOW(C) = \{d, e, \$ \}$

$FOLLOW(D) = \{e, \$ \}$

$FOLLOW(E) = \{\$ \}$

8

2) $S \rightarrow Bb/cd$

$B \rightarrow ab/\epsilon$

$C \rightarrow cC/\epsilon$

$FIRST(S) = \{a, b, c, d\}$

$FIRST(B) = \{a, \epsilon\}$

$FIRST(C) = \{c, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(B) = \{b\}$

$FOLLOW(C) = \{d\}$

3) $E \rightarrow TE'$

$E' \rightarrow +TE'/\epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\epsilon$

$F \rightarrow id/(C)$

$FIRST(E) = \{id, C\}$

$FIRST(E') = \{+, \epsilon\}$

$FIRST(T) = \{id, C\}$

$FIRST(T') = \{*, \epsilon\}$

$FIRST(F) = \{id, C\}$

$FOLLOW(E) = \{\$, \epsilon\}$

$FOLLOW(E') = \{\$, \epsilon\}$

$FOLLOW(T) = \{+, \$, \epsilon\}$

$FOLLOW(T') = \{+, \$, \epsilon\}$

$FOLLOW(F) = \{*, +, \$, \epsilon\}$

4) $S \rightarrow ACB/\epsilon bB/Ba$

$A \rightarrow da/BC$

$B \rightarrow g/\epsilon$

$C \rightarrow h/\epsilon$

$FIRST(S) = \{\epsilon, d, g, h, b, a\}$

$FIRST(A) = \{d, g, h, \epsilon\}$

$FIRST(B) = \{g, \epsilon\}$

$FIRST(C) = \{h, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{h, g, \$ \}$

$FOLLOW(B) = \{\$, a, h, g\}$

$FOLLOW(C) = \{g, \$, b, h\}$

5) $S \rightarrow aABb$

$A \rightarrow C/\epsilon$

$B \rightarrow d/\epsilon$

$FIRST(S) = \{a\}$

$FIRST(A) = \{C, \epsilon\}$

$FIRST(B) = \{d, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{d, b\}$

$FOLLOW(B) = \{b\}$

6) $S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC/\epsilon$

$D \rightarrow Ef$

$E \rightarrow g/\epsilon$

$F \rightarrow f/\epsilon$

$FIRST(S) = \{a\}$

$FIRST(B) = \{c\}$

$FIRST(C) = \{b, \epsilon\}$

$FIRST(D) = \{g, b, \epsilon\}$

$FIRST(E) = \{g, \epsilon\}$

$FIRST(F) = \{f, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(B) = \{g, b, h\}$

$FOLLOW(C) = \{g, b, h\}$

$FOLLOW(D) = \{h\}$

$FOLLOW(E) = \{f, h\}$

$FOLLOW(F) = \{h\}$

17

$E \rightarrow TE'$

$E' \rightarrow +TE'/\epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\epsilon$

$F \rightarrow id/(C)$

NOW THE

	id
E	$E \rightarrow TE$
E'	
T	$T \rightarrow FT$
T'	
F	$F \rightarrow$

2) $S \rightarrow (S)$

S	s
---	---

Now, $w =$

CONSTRUCTION OF LL(1) PARSING TABLE (L-7)

9.

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{id, c\}$$

$$\text{FOLLOW}(E) = \{\$, \}$$

$$E' \rightarrow +TE' / \epsilon$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FOLLOW}(E') = \{\$, \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{id, c\}$$

$$\text{FOLLOW}(T) = \{+, \$, \}$$

$$T' \rightarrow *FT' / \epsilon$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

$$\text{FOLLOW}(T') = \{+, \$, \}$$

$$F \rightarrow id / (E)$$

$$\text{FIRST}(F) = \{id, c\}$$

$$\text{FOLLOW}(F) = \{*, +, \$, \}$$

NOW THE LL(1) PARSING TABLE LOOKS LIKE

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

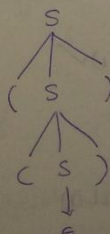
$$\Rightarrow S \rightarrow (S) / \epsilon$$

	(	)	\$
S	$S \rightarrow (S)$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

All Grammars are not feasible for LL(1) parsing.

$$\text{Now, } w = (())\$$$

$\$ \mid S \mid (\mid) \mid \$ \mid$



ANY GRAMMAR WHICH IS LEFT RECURSIVE / NO CANNOT BE USED FOR LL(1) PARSING

Non-deterministic

CHECK WHETHER THE GRAMMARS ARE LL(1) OR NOT

$$1) S \rightarrow aSbS / bSaS / \epsilon$$

$$\{a\} \quad \{b\} \quad \{a, b, \$\} \Rightarrow \text{Not LL(1)}$$

↳ which means the production must be placed under 's' row and 'a' column.

↳ Since the $S \rightarrow \epsilon$ production should be placed under the follow(s) which are $\{a, b\}$, we have already placed production in 's' row and 'a' column which means a single CELL has more than one entry, so the grammar is not LL(1) X

$$2) S \rightarrow aABb \Rightarrow \{a\} \quad A \rightarrow c \quad \rightarrow A \rightarrow \epsilon$$

$$A \rightarrow c / \epsilon \Rightarrow \{c\} \text{ and follow}(A) = \{d, b\}$$

$$B \rightarrow d / \epsilon \Rightarrow \{d\} \text{ and } \{b\} \Rightarrow \text{IS LL(1)} \checkmark$$

$$3) S \rightarrow A/a \quad \{a\} \cap \{a\} = \text{Not LL(1)} \quad X$$

↳ Conflict

$$A \rightarrow a \quad \{a\}$$

$$4) S \rightarrow aB / \epsilon \quad \{a\} \quad \{ \$ \} \Rightarrow \text{Not } x^n$$

$$B \rightarrow bC / \epsilon \quad \{b\} \quad \{ \$ \} \Rightarrow \text{NO } x^n$$

$$C \rightarrow cS / \epsilon \quad \{c\} \quad \{ \$ \} \Rightarrow \text{NO } x^n$$

} LL(1) \checkmark

$$5) S \rightarrow AB$$

$$A \rightarrow a / \epsilon \Rightarrow \{a\} \quad \{b, \$\} \Rightarrow \text{NO } x^n$$

$$B \rightarrow b / \epsilon \Rightarrow \{b\} \quad \{ \$ \} \Rightarrow \text{NO } x^n$$

} LL(1) \checkmark

$$6) S \rightarrow aSA / \epsilon \Rightarrow \{a\} \quad \{c, \$\} \Rightarrow \text{NO } x^n$$

$$A \rightarrow c / \epsilon \Rightarrow \{c\} \quad \{c\} \Rightarrow x^n \text{ is there} \quad \} X$$

$$7) S \rightarrow A$$

$$A \rightarrow Bb / cd \Rightarrow \{a, b\} \quad \{c, d\}$$

$$B \rightarrow aB / \epsilon \Rightarrow \{a\} \quad \{b\}$$

$$C \rightarrow cC / \epsilon \Rightarrow \{c\} \quad \{d\}$$

} LL(1) \checkmark

$$9) S \rightarrow iEtss/a \quad \{i\} \quad \{a\}$$

$$s' \rightarrow es / \epsilon \quad \{e\} \quad \{e\}$$

$$E \rightarrow b$$

} Not LL(1) X

$$8) S \rightarrow aAa / \epsilon \quad \{a\} \quad \{ \$, a\} \Rightarrow \text{Not LL(1)}$$

$$A \rightarrow abS / \epsilon$$

RECURSIVE

$$E \rightarrow iE'$$

$$E' \rightarrow +iE'/\epsilon$$

⇒ The parser we are going

$$E()$$

$$1 \text{ if } (l =$$

$$2 \{$$

$$\{^4$$

$$l = \text{getchar}()$$

main()

$$\{$$

$$1) E()$$

$$2) \text{ if } (l =$$

$$3) \text{ printf}$$

$$\}$$

OPERATOR

OPERATOR

A Grammar operator G

⇒ NO Two

⇒ NO eps

⇒ This p

RECURSIVE DECENT PARSER (L-8)

$E \rightarrow iE'$
 $E' \rightarrow +iE'/\epsilon$

The parser is called Recursive Decent parser because for every variable we are going to write Recursive functions.

```
E()
{
    1) if (l == 'i')
    2) {
        match('i');
    }
    3) E'();
}
l = getch();
```

```
E'()
{
    1) if (l == '+')
    2) {
        match('+');
    }
    3) match('i');
    4) E'();
}
else return;
```

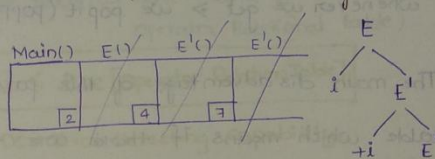
```
match(char t)
{
    if (l == t)
        l = getch();
    else
        printf("error");
}
```

main()

```
{
    1) E()
    2) if (l == '$')
    3) printf("parsing success");
}
```

⇒ This parser uses Recursion stack for parsing.
 ⇒ Here l = look Ahead

{i + i \$} is generated from above grammar



OPERATOR GRAMMAR AND OPERATOR PRECEDENCE PARSER (L-9)

OPERATOR GRAMMAR

A Grammar that is used to define the mathematical operations is called operator Grammar (with some Restrictions)

- ⇒ NO Two variables must be Adjacent
- ⇒ No Epsilon productions

$E \rightarrow E + E / E * E / id (\checkmark)$

$E \rightarrow EAE / id$
 $A \rightarrow + / *$

} Not operator Grammar

$S \rightarrow SAS / a$
 $A \rightarrow bsb / b$

Not operator precedence

$S \rightarrow Sbsbs / sb / a$
 $A \rightarrow bsb / b$

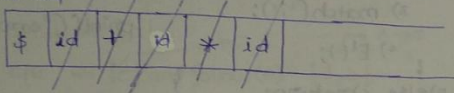
operator precedence Grammar.

⇒ This parser parses the Ambiguous grammars by creating operator Relation

The operator Relational table looks like

	id	+	*	\$
id	-	>	>	>
+	<	>	<	<
*	<	>	>	>
\$	<	<	<	=

W = id + id * id \$
 ↑ ↑ ↑ ↑ ↑
 look head



Now The Algorithm goes like this

- 1) when the top of the stack is < than the lookAhead then push it and whenever we get > we pop it (popping means actually we Reduced it)

The main disadvantage of this parser is the size of the operator Relational table which means if there are 4 operators then size of the table is 16 (4²) and if there are 5 operators then there would be 25 entries (5²). So Generally if there are 'n' operators, the size of the table is $O(n^2)$

Now, To reduce the size of the table we use operator function table

	id	+	*	\$
id	-	>	>	>
+	<	>	<	<
*	<	>	>	>
\$	<	<	<	<

↓

Function(F)

> = F_{id} → G_{id} (F → G)

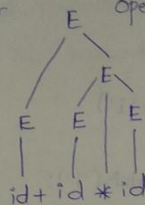
< = G_{id} → F_{id} (G → F)

The given grammar is

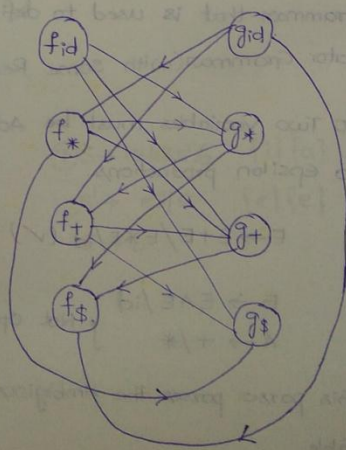
$E \rightarrow E + E / E * E / id$

⇒ id will have higher precedence than any other operator

⇒ '\$' will have least precedence than any other operator



⇒ Top of the stack will be \$.



Now, the longest path from f_{id} is $f_{id} \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$$ 13
 Now, the longest path from g_{id} is $g_{id} \rightarrow f_* \rightarrow g_+ \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$$
 Similarly find the longest paths from each node the function table looks like

	id	+	*	\$
f	4	2	4	0
g	5	1	3	0

$f_{id} \xrightarrow{1} g_* \xrightarrow{2} f_+ \xrightarrow{1} g_+ \xrightarrow{4} f_\$ = 4$
 $\rightarrow f_+ \xrightarrow{1} g_+ \xrightarrow{2} f_\$ = 2$

Now if indeed to compare $(+, +) \Rightarrow f_+ \quad g_+$
 $\Rightarrow 2 \quad 1 \Rightarrow 2 > 1 \Rightarrow (+ > +)$

$\therefore$ The Size of the table = $O(2n)$ n = no. of operators.

$\Rightarrow$ In the functional table we don't have Blank entries (Blank Entries are nothing but errors) so, the error detecting capability of the functional table is less than that of operator Relation table (we have Blank entries in operator Relational table)

$$EDC [\text{Operator Functional Table}] < EDC [\text{Operator Relation Table}]$$

EDC = Error Detecting Capability.

2) $P \rightarrow SR/S$

$R \rightarrow bSR/bs$

$S \rightarrow wbs/w$

$w \rightarrow L*w/L$

$L \rightarrow id$

$P \rightarrow SbP/Sbs/S$

$S \rightarrow wbs/w$

$w \rightarrow L*w/L$

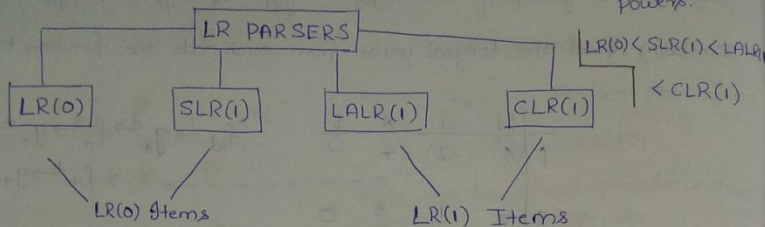
$L \rightarrow id$

	id	*	b	\$
id	-	>	>	>
*	<	<	>	>
b	<	<	<	>
\$	<	<	<	-

$\Rightarrow$ Here $*$ is defined as Right Associative ($w \rightarrow L*w$) so the Right side star has highest precedence
 $\therefore * < *$

LR PARSING, LR(0) ITEMS AND LR(0) PARSING TABLE (L-10)

14



1) $S \rightarrow AA$
 $A \rightarrow aA/b$ } In LR parsers we have CLOSURE and GOTO Operations

The Augmented Grammar is

$S' \rightarrow S$
 $S \rightarrow AA$
 $A \rightarrow aA/b$

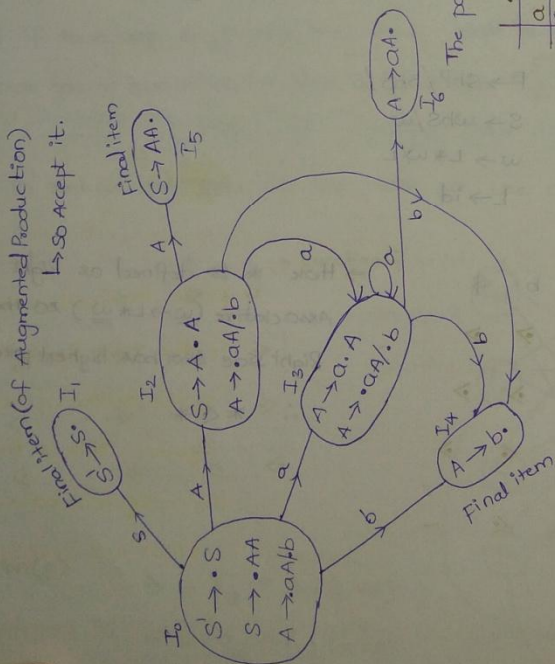
Any production with a dot in the RHS is called an item.

The LR(0) parsing Tree is,

$S' \rightarrow S$
 $S \rightarrow AA$ ①
 $A \rightarrow aA/b$ ② ③

The parsing table is

ACTION	GOTO					
	a	b	\$	A	S	
0 S_3	1 S_3	2 S_4	Accept	5	6	
1 S_3	2 S_4	3 S_3				
2 S_4	3 S_3	4 R_3				
3 S_3	4 R_3	5 R_1				
4 R_3	5 R_1	6 R_2				
5 R_1	6 R_2					
6 R_2						



1. while writing the Reduce moves / while inspecting final items go to the productions and check

$S' \rightarrow S$
 $S \rightarrow AA$ ①
 $A \rightarrow aA$ ②
 $A \rightarrow b$ ③

Now I_4 is $A \rightarrow b \cdot$ (3rd production)
 $\Rightarrow I_4 \rightarrow R_3$

LR(0) PARSING

$S' \rightarrow S$
 $S \rightarrow AA$ ①
 $A \rightarrow aA/b$ ② ③

Input

$\Rightarrow$ Always the
 $\Rightarrow$ Initially 'I'
 at and as
 and increm
 $\Rightarrow$ Now top of
 which stat
 previous
 $\Rightarrow$ Now In th
 RHS of the
 RHS. In thi
 which me
 'x' then
 onto the
 it turns
 = 6, 50
 SLR(0)

$R_1: S \rightarrow AA$
 place R_1 in follow
 follow(S) = { $\$$ }

LR(0) PARSING EXAMPLE AND SLR(1) TABLE (L=1)

15

$S' \rightarrow S$
Let the given string be aabb\$

$S \rightarrow AA$

$A \rightarrow aA/b$

② ③

Input pointer $\uparrow \uparrow \uparrow \uparrow$ aabb\$

0	a	3	a	3	b	4	A	6	A	6	A	2	b	4	A	5	S	1	Accept
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--------

→ Always the top of the stack contains state (and first state will be Zero)

→ Initially 'I₀' on 'a' is 'S₃' which means shift the input you are looking at and as well as the state no on to the stack $\Rightarrow$ [Input=a, State=3] = a|3 and increment the Input pointer [continue]

→ Now top of the stack is '4' and Input pointer is b which means 'R₃' which states that Reduce the production no. 3. which means reduce the previous 'b'. (previous symbol). $\hookrightarrow (A \rightarrow b)$

→ Now in the stack how could we make Reduce move is look the RHS of the production that must be Reduced, and find the length of RHS. In this example 'A $\rightarrow$ b' is the production $\Rightarrow$ length of RHS = 1 which means pop 2 symbols (1x2) from stack. (If the length of RHS is 'x' then pop 2x' elements from stack) and push the LHS symbol onto the stack, and see the stack for the last used state number it turns out that it is '3' and look what '3' on 'A' is generating = 6, so push '6' onto the stack. when we see reduce moves we don't increment Input pointer.

SLR(1)

	ACTION			GOTO	
	a	b	\$	A	S
0	S ₃	S ₄		2	1
1			Accept		
2	S ₃	S ₄		5	
3	S ₃	S ₄		6	
4	R ₃	R ₃	R ₃		
5					
6	R ₂	R ₂	R ₂		

R₁: S $\rightarrow$ AA

Place R₁ in follow(s)

follow(s) = { \$ }

place the Reduce moves

if the next Symbol is

in the follow of current

Symbol, this is the

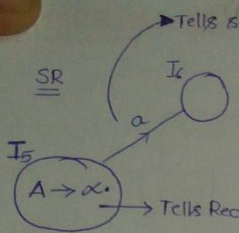
main difference between the

LR(0) and SLR(1) parsers

Now,

R₃ $\Rightarrow$ A $\rightarrow$ b should be reduced

place R₃ in follow(A) = { a, b, \$ }

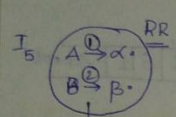


LR(0)
 ⇒ All the Grammars are not suitable for LR parsing due to shift Reduce conflicts.

	a	b	c
5	r1/r2	r1/r2	r1/r2

	a	b
5	r1/r2	r1

SR-CONFLICT

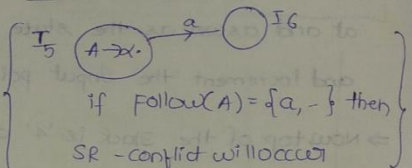


⇒ A Grammar cannot be LR(0) if it have SR/RR conflicts.

EXAMPLES OF LR(0) AND SLR(1) (L-12)

- 1) $S \rightarrow dA/aB \{d, a\}$
- 2) $A \rightarrow bA/c \{b, c\}$
- 3) $B \rightarrow bB/c \{b, c\}$

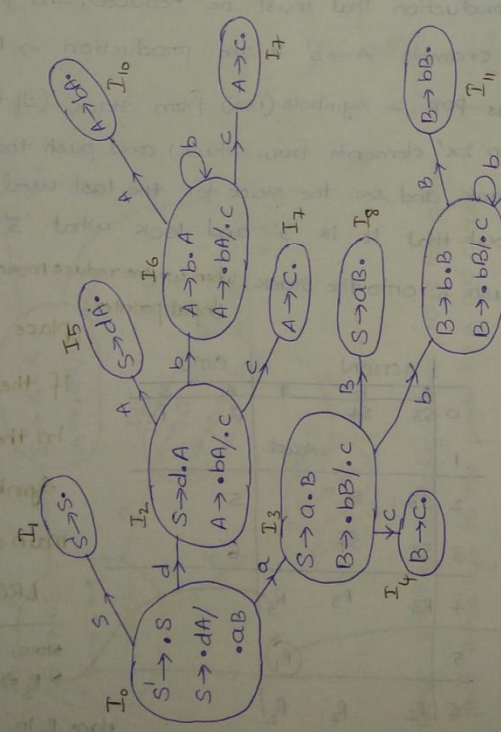
- 1) LL(1) ✓
- 2) LR(0) ✓
- 3) SLR(1) ✓



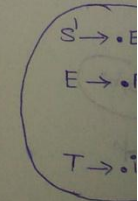
③ $S \rightarrow (L)/$
 $L \rightarrow L,S$

CANONICAL

CANONICAL COLLECTION OF LR(0) ITEMS



④ $E \rightarrow E + T$
 $T \rightarrow i$



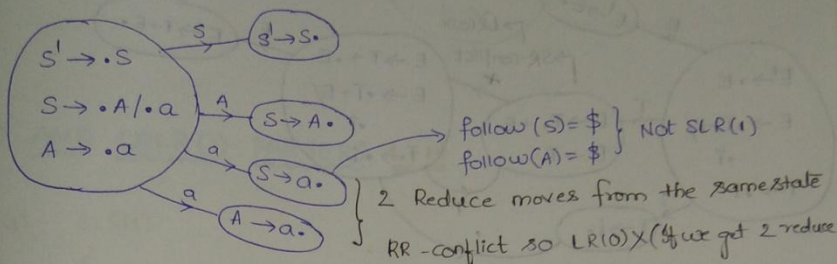
$S \rightarrow A/a \{a\} \{a\}$ Not LL(1) x
 $A \rightarrow a$ LR(0) x
 SLR(1) x

} Ambiguous Grammar

parsing

- CONFLICT

SR / RR



moves from the same state then
the grammar is not LR(0).

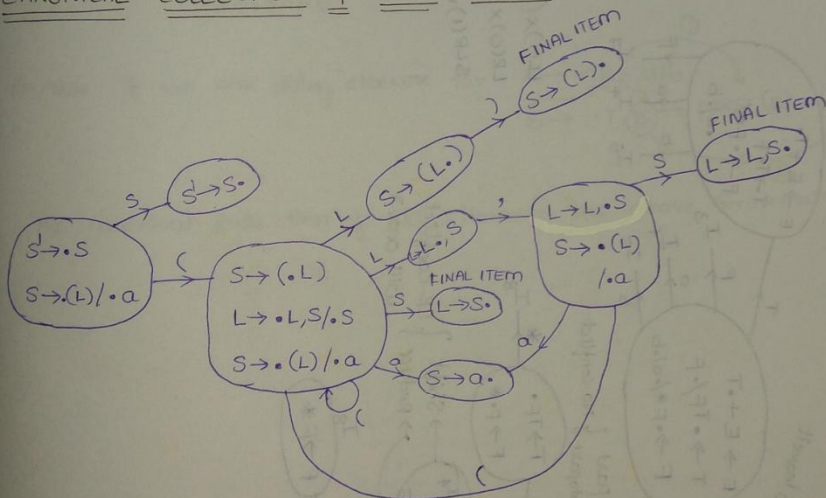
③ $S \rightarrow (L)/a$
 $L \rightarrow L, S/S$

LL(1) $\times$ (Left Recursive)

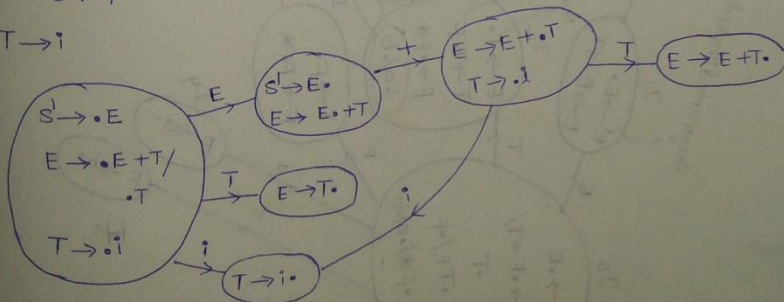
 $LR(0) \checkmark$
$$\Rightarrow \text{sol}$$

SLR(1) ✓

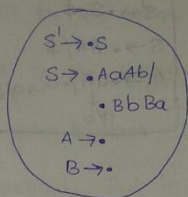
CANONICAL COLLECTION OF LRCO ITEMS



④ $E \rightarrow E + T / T$

$$T \rightarrow i$$


77 $S \rightarrow AaAb / BbBa \{a, b\}$ LL(1) ✓
 $A \rightarrow \epsilon$ LR(0) X
 $B \rightarrow \epsilon$ SLR(1) X



	a	b	\$
0	$\frac{r_3}{r_3/r_4}$	$\frac{r_3}{r_3/r_4}$	

RR- conflict

CLR(1) AND LALR(1) PARSERS L-14

LALR(1) CLR(1)

LR(1) Items = LR(0) Items + lookahead

① $S \rightarrow AA$

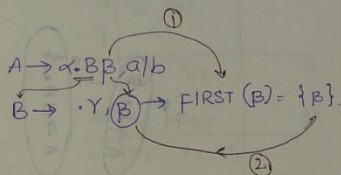
$A \rightarrow aA/b$

$S' \rightarrow \cdot S$ is the Augmented Grammar

$S \rightarrow \cdot AA$

$A \rightarrow \cdot aA / \cdot b$

In case if we are doing closure for



The canonical collection of LR(1) Items for the above Grammar is

Augmented Grammar

$S \rightarrow \cdot S, \$$
 $S \rightarrow \cdot AA / \$$
 $A \rightarrow \cdot aA / \cdot b \rightarrow a/b$
 $\quad \quad \quad \downarrow$
 $\quad \quad \quad a/b$

lookahead is always \$ for Augmented production.

(20)

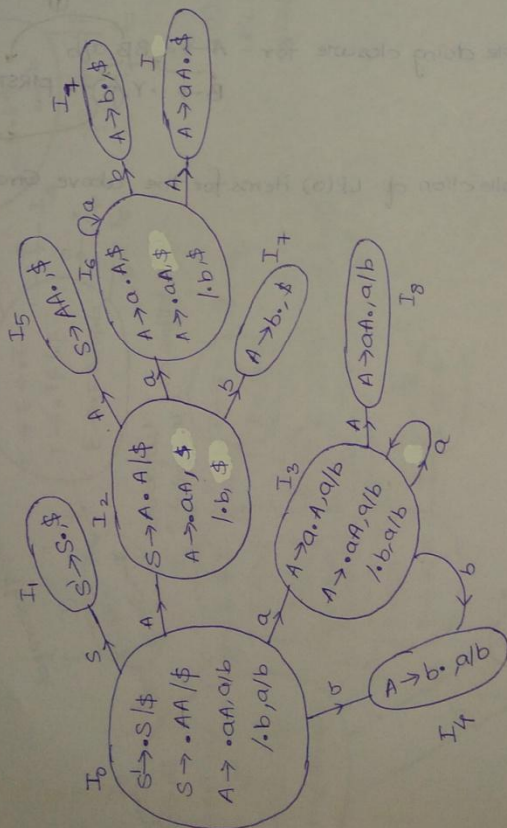
CLR(1) AND LALR(1) PARSERS

LR(0) Items = LR(0) Items + Lookahead

⇒ The Goto
 tables,
 In the
 the fol
 are g
 ⇒ from th
 look head
 ⇒ Similar
 Look ab

CLR(1) (PA

CANONICAL COLLECTION OF LR(1) ITEMS



$I_3, I_6 \Rightarrow$

$I_4, I_7 \Rightarrow$

$I_8, I_9 \Rightarrow$

[No of sta

\$ for Augmented

(20)

⇒ The Goto part and shift part will be the same as LR(0), SLR(1) parsing tables, the main difference arises in the placement of the final items. In the LR(0) and SLR(1) we are going to place in the entire row and the follow of LHS (in SLR(1)) respectively. But in CLR and LALR(1) we are going to place the reduce moves only in look ahead symbols.

⇒ from the above diagram $[I_3, I_6]$ have same LR(0) items but differ in look heads

⇒ Similarly $[I_4, I_7]$, $[I_8, I_9]$ also have same LR(0) items but differ in look aheads.

CLR(1) PARISING TABLE

	a	b	\$	S	A
0	S ₃	S ₄			2
1					
2	S ₆	S ₇			5
3	S ₃	S ₄			8
4	r ₃	r ₃			
5			r ₁		
6	S ₆	S ₇			9
7			r ₃		
8	r ₂	r ₂			
9			r ₂		

LALR(1) TABLE

	a	b	\$	S	A
0	S ₃₆	S ₄₇			2
1					
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

$I_3, I_6 \Rightarrow I_{36}$

$I_4, I_7 \Rightarrow I_{47}$

$I_8, I_9 \Rightarrow I_{89}$

$[No \text{ of states in CLR(1)}] \geq [No \text{ of states in SLR(1)}] = [No \text{ of states in LALR(1)}] = [No \text{ of states in LR(0)}]$

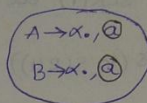
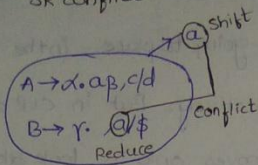
(21)

CONFLICTS AND EXAMPLES OF CLR(1) AND LALR(1) (L-15) (20)

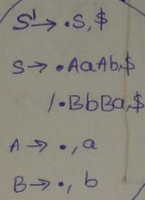
SR conflict

RR conflict

LR(1) Items

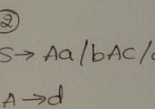
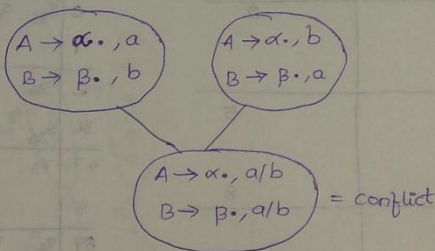


LR(0) Items



⇒ If the grammar is not CLR(1) then the Grammar is not LALR(1) because we reduce the size of the table but not the conflicts in LALR(1) parser

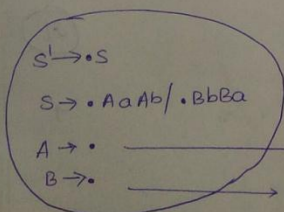
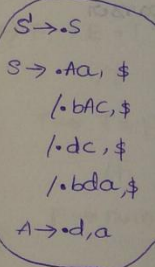
⇒ If the Grammar is CLR(1) then it may or maynot be LALR(1)



i) $S \rightarrow AaAb / BbBa$

$A \rightarrow \epsilon$
 $B \rightarrow \epsilon$

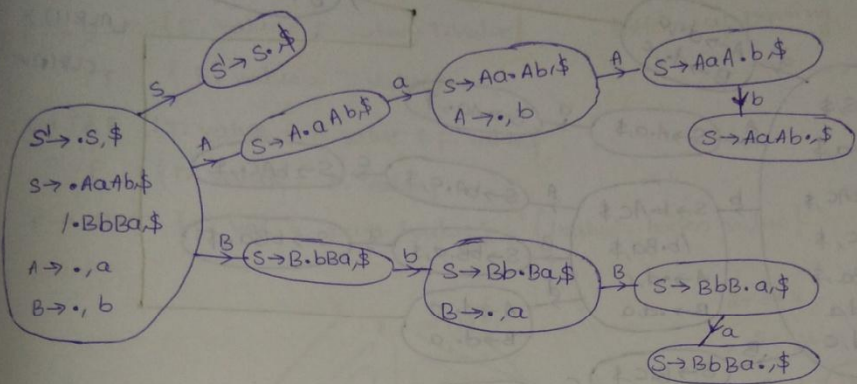
LL(1) X
LR(0) X
SLR(1) X
CLR(1) ✓
LALR(1) ✓



placed under follow of $A = \{a, b\}$
placed under follow of $B = \{a, b\}$

} ⇒ Not SLR(1)

22



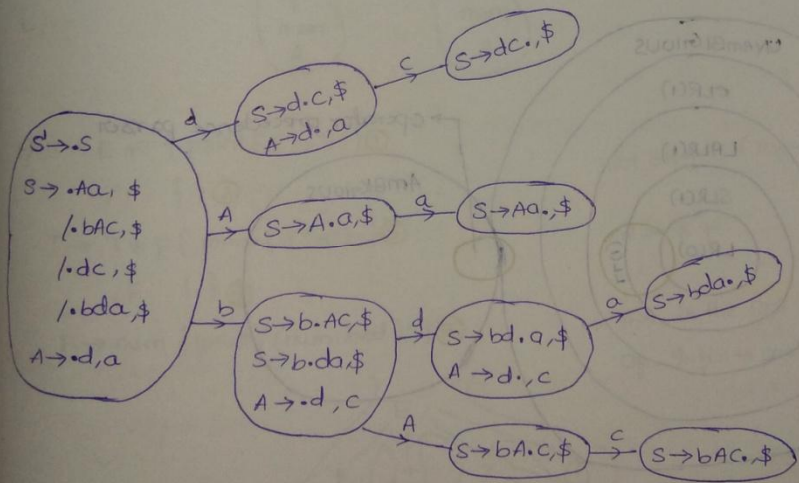
because
parser

③

$S \rightarrow Aa/bAc/dc/bda$

$A \rightarrow d$

- LL(1) X
- LR(0) X
- SLR(1) X
- CLR(1) ✓
- LALR(1)

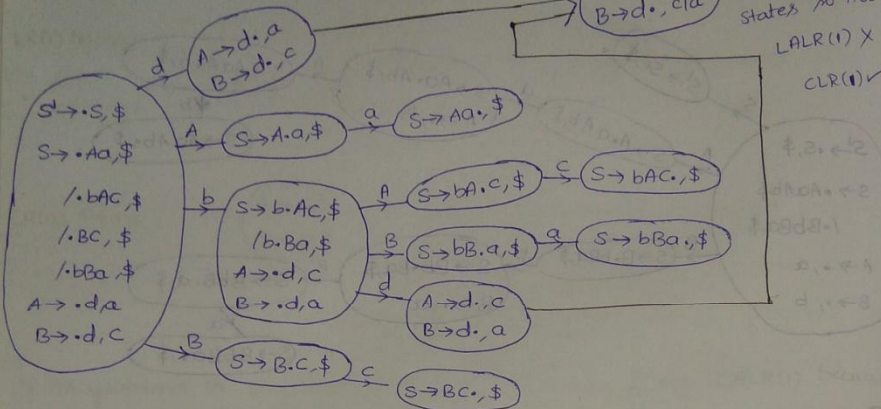


R(1)

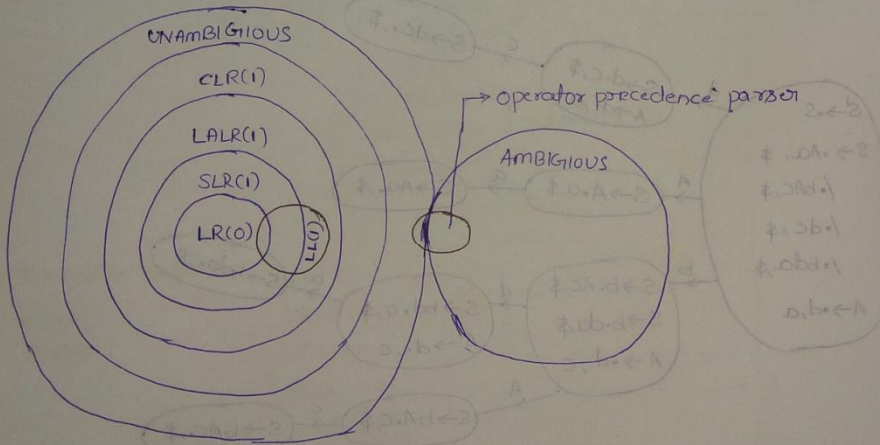
③ $S \rightarrow Aa/bAc/Bc/bBa$

$A \rightarrow d$

$B \rightarrow d$



COMPARISON OF THE PARSERS (L-16)



⇒ Every Grammar which is LL(1) is definitely LALR(1)

SYNTAX DIRE

⇒ Grammar +

1) $E \rightarrow E+T \{E, T\}$
 $T \rightarrow T * F \{T, F\}$
 $F \rightarrow \text{num} \{\text{num}\}$

2) $E \rightarrow E+T \{E, T\}$
 $T \rightarrow T * F \{T, F\}$
 $F \rightarrow \text{num} \{\text{num}\}$

SYNTAX DIRECTED TRANSLATION (L-17)

conflict arises if
we merge the
states so not

LALR(1) X

CLR(1) ✓

⇒ Grammar + Semantic Rules = SDT

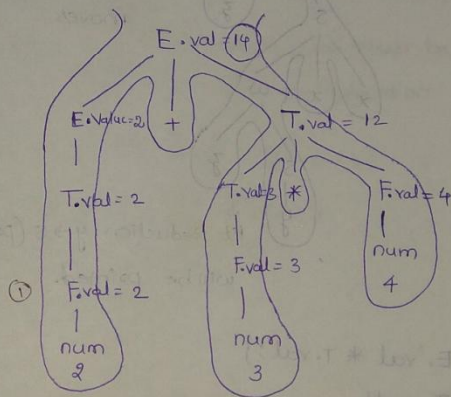
$$1) E \rightarrow E + T \quad \{ E.value = E.value + T.value \}$$

$$\quad \quad \quad / T \quad \{ E.value = T.value \}$$

Bottom-up parsing.

$$T \rightarrow T * F \quad \{T. \text{value} = T. \text{value} * F. \text{value}\}$$

/F {T. value = F. value}

$$F \rightarrow \text{num} \{ F.\text{val} = \text{num.lvalue} \} \quad \{ \text{lvalue} = \text{lexim value} \}$$


$$2 + 3 * 4$$

$$= 2 + (3 * 4)$$

$$= 2 + 12 = 14$$

$$2) E \rightarrow E + T \{ \text{printf} (" + "); \} \textcircled{1}$$

1/T f f (2)

$2 + 3 * 4$ (Top-down parser)

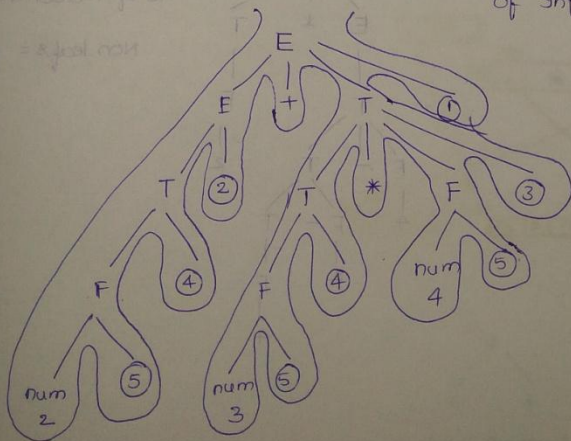
$$T \rightarrow T * F \{ \text{printf} ("*"); \} \textcircled{3}$$

234 * +

$$1/F \quad \{ \} \textcircled{4}$$

$F \rightarrow \text{num} \{ \text{printf}(\text{num.lval}); \}$ ⑤

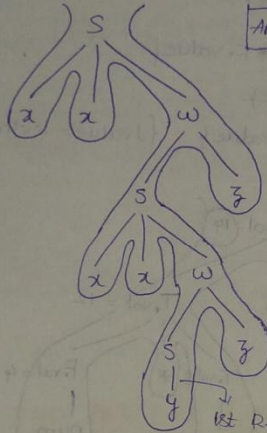
⇒ This is the SDT for conversion of Infix to postfix Expression



Top down
passing

③ $S \rightarrow xzw \{ \text{printf}(1); \}$
 $/y \{ \text{printf}(2); \}$
 $w \rightarrow Sz \{ \text{printf}(3); \}$

String $xxzxzyzz$



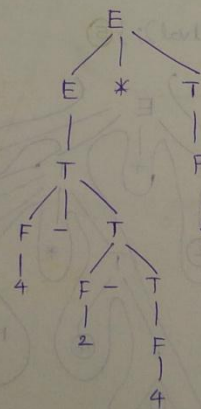
Ans: 2 3 1 3 1

$\Rightarrow$ Carry the action on Reduce moves

Reduction $y \rightarrow S(\text{printf}(2));$ '502' will be printed.

④ $E \rightarrow E * T \{ E.val = E.val * T.val; \}$
 $/T \{ E.val = T.val; \}$
 $T \rightarrow F - T \{ T.val = F.val - T.val; \}$
 $/F \{ T.val = F.val; \}$
 $F \rightarrow 2 \{ F.val = 2; \}$
 $/4 \{ F.val = 4; \}$

$$\begin{aligned} W &= 4 / 2 - (4 * 2) \\ W &= ((4 - 2) * 2) \\ &= (4 - 2) * 2 \\ &= (4 + 2) * 2 \\ &= 12 \end{aligned}$$



No of Reductions = No. of

Non leafs = 10

⑤ $E \rightarrow E \# T$
 $/T$
 $T \rightarrow T \& F$
 $/F$
 $F \rightarrow \text{num}$

$$\begin{aligned} W &= 2 \# 3 \& 5 \\ &= 2 * 3 + 5 \\ &= ((2 * 3) + 5) \\ &= (6 + 5) \\ &= 11 \end{aligned}$$

L-18

SDT TO BUILD

⑥ $E \rightarrow E_1 + T$
 $/T$
 $T \rightarrow T_1 * F$
 $/F$
 $F \rightarrow \text{id}$
 $W = 2 + 3 * 4$

$E_{\text{input}} = 100$
 $T_{\text{input}} = 100$
 $F_{\text{input}} = 100$

26

⑥ $E \rightarrow E \# T \{ E.val = E.val * T.val; \}$
 $/T \{ E.val = T.val; \}$
 $T \rightarrow T \& F \{ T.val = T.val + F.val; \}$
 $/F \{ T.val = F.val; \}$
 $F \rightarrow num \{ F.val = num.value; \}$

27

$w = 2 \# 3 \& 5 \# 6 \& 4$ what is the output

$= 2 * 3 + 5 * 6 + 4$
 $= ((2 * 3) + (5 * 6) + 4)$
 $= (6 + 30 + 4)$
 $= 40$

wrong because '+' is defined at Highest level (Bottom level) and must be evaluated first and then multiplication must be evaluated.

$\Rightarrow 2 * (3 + 5) * (6 + 4)$
 $= 2 * (8) * (10)$
 $= 160 \checkmark$

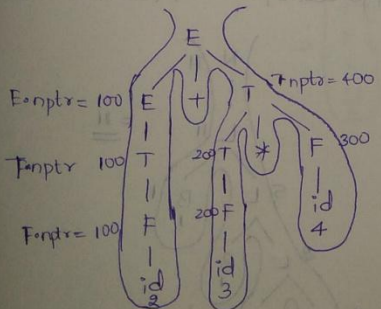
L-18

SDT TO BUILD SYNTAX TREE

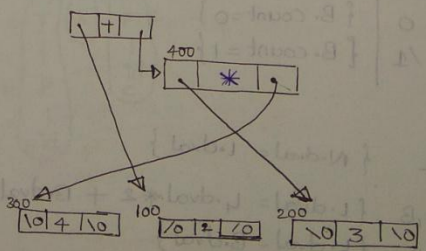
③ $E \rightarrow E_1 + T \{ E.nptr = mknode(E_1.nptr, '+', T.nptr); \}$
 $/T \{ E.nptr = T.nptr; \}$
 $T \rightarrow T_1 * F \{ T.nptr = mknode(T_1.nptr, '*', F.nptr); \}$
 $/F \{ T.nptr = F.nptr; \}$
 $F \rightarrow id \{ F.nptr = mknode(null, id.name, null); \}$

Returns Address
 $\uparrow$
 $mknode = make node$
 $nptr = node pointer$

$w = 2 + 3 * 4$



Concrete Syntax tree



Abstract Syntax tree

⑦ SDT FOR TYPE CHECKING

$E \rightarrow E_1 + E_2 \{ \text{if } ((E_1.type == E_2.type) \&\& (E_1.type == \text{int})) \text{ then } E.type = \text{int} \text{ else error} \}$

$/E_1 == E_2 \{ \text{if } ((E_1.type == E_2.type) \&\& (E_1.type == \text{int} / \text{boolean})) \text{ then } E.type = \text{boolean} \text{ else error} \}$

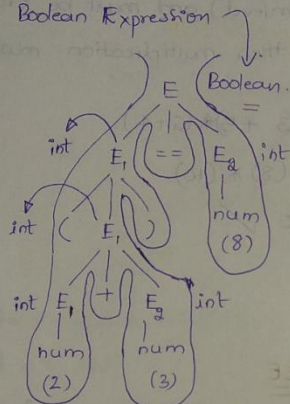
$(E_1) \{ E.type = E_1.type \}$

$/\text{num} \{ E.type = \text{int}; \}$

$/\text{true} \{ E.type = \text{boolean}; \}$

$/\text{false} \{ E.type = \text{boolean}; \}$

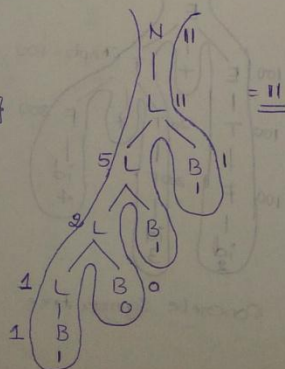
$w = \{ (2+3) == 8 \} = \text{Boolean Expression}$



	<u>count</u> <u>all</u> <u>1's</u>
$N \rightarrow L$	$\{ N.count = L.count \}$
$L \rightarrow L_1 B$	$\{ L.count = L_1.count + B.count \}$
$/B$	$\{ L.count = B.count \}$
$B \rightarrow 0$	$\{ B.count = 0 \}$
$/1$	$\{ B.count = 1 \}$

<u>count</u> <u>all</u> <u>1's</u>	No. of B's
$\Rightarrow$	$\Rightarrow$
$\Rightarrow$	$\Rightarrow$
$\Rightarrow$	$\Rightarrow$
$\{ B.count = 1 \}$	$\{ B.count = 1 \}$
$\{ B.count = 0 \}$	$\{ B.count = 1 \}$

$N \rightarrow L$	$\{ N.dval = L.dval \}$
$L \rightarrow L_1 B$	$\{ L.dval = L_1.dval * 2 + B.dval \}$
$/B$	$\{ L.dval = B.dval \}$
$B \rightarrow 0$	$\{ B.dvalue = 0 \}$
$/1$	$\{ B.dvalue = 1 \}$



$w = 1011$

L-19 S-ATT

⑩ $S \rightarrow id = E$

$N \rightarrow L_1 L_2$

$L \rightarrow LB / B$

$B \rightarrow 0$

$/1$

S D T T O G E N E

⑩ $S \rightarrow id = E$

$E \rightarrow E_1 + T$

$/T$

$T \rightarrow T_1 * F$

$/F$

$F \rightarrow id$

L-19 S- ATTRIBUTED AND L- ATTRIBUTED DEFINITIONS

(21)

① $\frac{1}{2} = 0.25$

② $\frac{3}{4} = 0.75$

③ $N \rightarrow L_1 \cdot L_2$
 $L \rightarrow L_1 B / B$
 $B \rightarrow 0 / 1$

$\{L_1.dval = L_1.dval + (L_2.dval / 8^{L_2.count})\}$
 $\{L_1.dval = L_1.dval + B.dval\} \{L_1.count = L_1.count + B.count\}$
 $\{L_1.count = B.count\} \{L_1.dval = B.dval\}$
 $\{B.count = 0, B.dval = 0\}$
 $\{B.count = 1, B.dval = 1\}$

SPT TO GENERATE THREE ADDRESS CODE

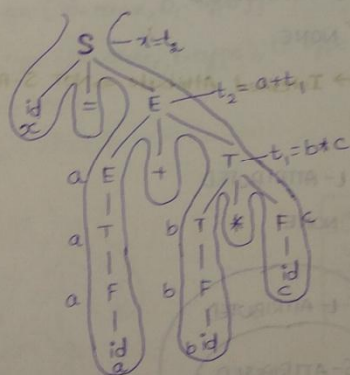
④ $S \rightarrow id = E \quad \{gen(id.name = E.place);\}$

$E \rightarrow E_1 + T \quad \{E.place = newTemp(); gen(E.place = E_1.place + T.place);\}$
 $/T \quad \{E.place = T.place;\}$

$T \rightarrow T_1 * F \quad \{T.place = newTemp(); gen(T.place = T_1.place * F.place);\}$
 $/F \quad \{T.place = F.place;\}$

$F \rightarrow id \quad \{F.place = id.name;\}$

$w \Rightarrow x = a + b * c$



$t_1 = b * c$
 $t_2 = a + t_1$
 $x = t_2$

DIFFERENCE BETWEEN S-ATTRIBUTED AND L-ATTRIBUTED SDT

S-ATTRIBUTED SDT

- 1) Uses only synthesized attributes
- 2) Semantic actions are placed at Right end of production
 $A \rightarrow BC \{ \}$
- 3) Attributes are evaluated during Bottom up parsing

L-ATTRIBUTED SDT

- 1) Uses Both inherited and Synthesized attributes. Each inherited attribute is restricted to inherit either from parent or Left siblings only.

Ex: $A \rightarrow XYZ \{ y.S = A.S, y.S = X.S, y.S = Z.S \}$

- 2) Semantic Actions are placed anywhere on RHS

$A \rightarrow \{ \} BC$
 $/D \{ \} E$
 $/FG \{ \}$

- 3) Attributes are evaluated by traversing parse tree depth first left to Right

Inherited Attribute

Not S-ATTRIBUTED

① $A \rightarrow LM \{ L.i = f(A.i); M.i = f(L.S); A.S = f(M.S); \}$

$A \rightarrow QR \{ R.i = f(A.i), Q.i = f(R.i); A.S = f(Q.S); \}$

NOT L-ATTRIBUTED

(A) S-ATTRIBUTED

(B) L-ATTRIBUTED

(C) BOTH

☒ NONE

② $A \rightarrow BC$

$\{ B.S = A.S \}$

Inherited Attribute $\Rightarrow$ NOT S-ATTRIBUTED

a) S-ATTRIBUTED

☒ b) L-ATTRIBUTED

(C) BOTH

(D) NONE



(30)

ED SDT

PT

nd Synthesized
d attribute is
either from parent

 $S = X.S, Y.S = Z.S$

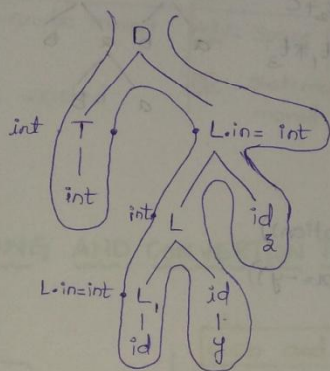
placed anywhere

ted by traversing
ft to Right

SDT TO ADD TYPE INFORMATION INTO SYMBOL TABLE

- ① $D \rightarrow TL \{L.in = T.type\} \Rightarrow$ Inherited Attribute
- $T \rightarrow \text{int} \{T.type = \text{int};\} \Rightarrow$ Synthesized Attribute
- $\quad \quad \quad / \text{char} \{T.type = \text{char};\}$
- $L \rightarrow L_1 id \{L_1.in = L.in, \text{add type}(id.name, L.in);\}$
- $\quad \quad \quad / id \{\text{add type}(id.name, L.in) \xrightarrow{L} \text{Inherited}\}$
- L-ATTRIBUTED
 $\text{int } x, y, z;$

x	int
y	int
z	int

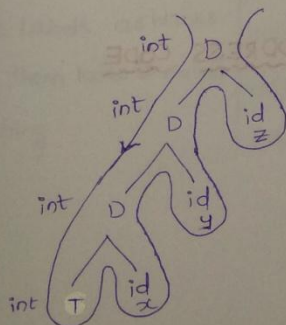


$\Rightarrow$ Evaluate the synthesized attribute when you last visit it

$\Rightarrow$ Evaluate the Inherited attribute when you first visit it.

S-ATTRIBUTED SDT FOR THE SAME QUESTION

- ② $D \rightarrow D_1 id \{\text{add-type}(id.name, D_1.type)\}$
- $\quad \quad \quad / T id \{\text{add-type}(id.name, T.type), D.type = T.type\}$
- $T \rightarrow \text{int} \{T.type = \text{int};\}$
- $\quad \quad \quad / \text{char} \{T.type = \text{char};\}$



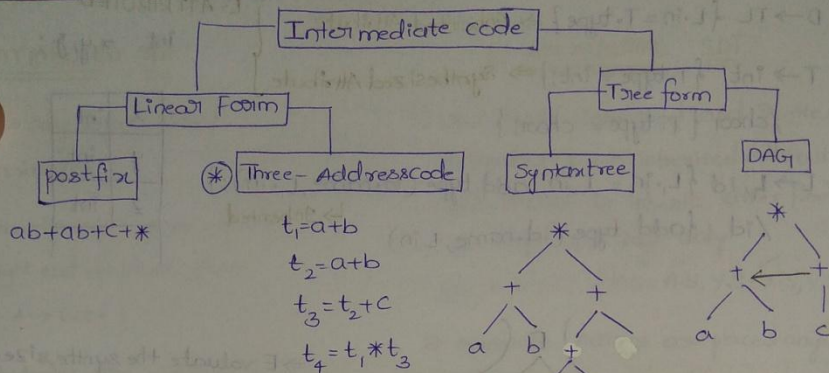
x	int
y	int
z	int

(31)

INTERMEDIATE CODE GENERATION

INTRODUCTION TO INTERMEDIATE CODE

Ex: $(a+b)*(a+b+c)$



TYPES OF 3-ADDRESS CODE

- 1) $x = y \text{ op } z$ ($x = a + b$ (Binary operation))
- 2) $x = \text{op } z$ (Unary operation ($x = -y$))
- 3) $x = y$ (Assignment)
- 4) if x (rel op) y goto L (if x (relational operator) y goto L)
- 5) goto L $\Rightarrow$ (unconditional)
- 6) $A[i] = x$ (Array indexing)
 $y = A[i]$
- 7) $x = *p \Rightarrow$ pointer
 $y = \&y \Rightarrow$ Address of variable assigned to another variable

VARIOUS REPRESENTATIONS OF 3-ADDRESS CODE

$\Rightarrow (a+b)*(c+d)+(a+b+c)$

1) $t_1 = a+b$

2) $t_2 = -t_1$

3) $t_3 = c+d$

4) $t_4 = t_2 * t_3$

5) $t_5 = a+b$

6) $t_6 = t_5 + c$

7) $t_7 = t_4 + t_6$

(32)

QUADRAPE

Op _r	Op ₁	Op ₂	Result
1) +	a	b	t ₁
2) -	t ₂	NULL	t ₂
3) +	c	d	t ₃
4) *	t ₂	t ₃	t ₄
5) +	a	b	t ₅
6) +	t ₅	c	t ₆
7) +	t ₄	t ₆	t ₇

Adv: Statements can be moved Around

Dis: More Space wasted

TRIPLE

Op _r	Op ₁	Op ₂
1) +	a	b
2) -	(1)	
3) +	c	d
4) *	(2)	(3)
5) +	a	b
6) +	(5)	c
7) +	(4)	(6)

Adv: Space is not wasted

Dis: Statements cannot be moved

INDIRECT TRIPLE (33)

Op _r	Op ₁	Op ₂
i) (1)		
ii) (2)		
iii) (3)		
iv) (4)		
v) (5)		
vi) (6)		
vii) (7)		

Adv: State ments can be moved

Dis: Two Access of Memory.

BACK PATCHING AND CONVERSION TO 3-ADDRESS CODEBack Patching

if (a < b) then t = 1
else t = 0

(i): if a < b goto (i+3)

(i+1): t = 0

(i+2): goto (i+4)

(i+3): t = 1

(i+4): Return.

Leaving the Labels as blanks and filling them later is called

Back patching

t₁ t₂ t₃
a < b and c < d or e < f

100) if (a < b) goto 103 110) goto 112

101) t₁ = 0

111) t₃ = 1

102) goto 104

112) t₄ = [t₁ and t₂]

103) t₁ = 1

113) t₅ = [t₄ or t₃]

104) if (c < d) goto 107

105) t₂ = 0

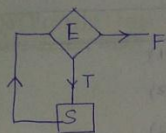
106) goto 108

107) t₂ = 1

108) if (e < f) goto 111

109) t₃ = 0

① While: E do S



L: if (E==0) goto LI (or) if (E) goto LI

S
Goto L

LI: S
goto L

② while (a<b) do
x = y + z

L: if a < b goto LI
goto last

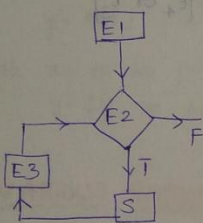
LI: t = y + z

x = t

goto L

last:

③ for (E1; E2; E3)



for (i=0; i<10; i++)

t = a + b + c;

i = 0

L: if (i < 10) goto LI

goto last

LI: t₁ = b + c

a = t₁

t = i + 1

i = t

goto L

last:

TWO

x = A

t₁ = y

t₂ = t₁

t₃ = t

t₄ = B

x =

L > B

Row M

column

(34)

② Switch (i+j)

{

case (i) = $a+b+c$;

break;

case (ii): $p=Q+R$;

break;

default: $x=y+z$;

break;

}

 $t = i+j$

goto test

L1: $t_1 = b+c$ $Q = t_1$

goto last

L2: $t_2 = Q+R$ $p = t_2$

goto last

L3: $t_3 = y+z$ $x = t_3$

goto last

test: if $(t==1)$ goto L1if $(t==2)$ goto L2

goto L3

last:

(35)

TWO DIMENSIONAL ARRAY TO 3-ADDRESS CODE $x = A[y][z]$

A: 10x20

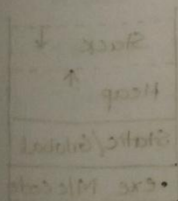
 $t_1 = y * 20$ $t_2 = t_1 + z$ $t_3 = t_2 * 4$ $t_4 = \text{Base Address of A}$ $x = t_4[t_3]$

→ Basic Offset Addressing

(Base Address + Offset)

Row Major order: 00 01 02 03 10 11 12 13 20 21 22 23 30 31 32 33

column major order: 00 10 20 30 01 11 21 31 02 12 22 32 03 13 23 33



A[4][4]

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

cross 2' row

A[2][3] columns

→ No of elements = $2 * 4 + 3$ in row $= 11$

RUN ENVIRONMENT

⇒ Run time Environment means when you run the program what is the support that you need from the operating system.

STORAGE ALLOCATION STRATEGIES

1) Static

- 1) Allocation is done at compile time
- 2) Bindings do not change at Runtime
- 3) One Activation Record per procedure

Disadvantages

- 1) Recursion is not supported.
- 2) Size of data objects must be known at compile time
- 3) Data structures cannot be created dynamically

2) Stack

whenever a new activation begins, Activation record is pushed onto the Stack and whenever Activation ends, Activation record is popped off

⇒ Local variables are bound to fresh storage

Disadvantages

- 1) Local variables cannot be retained once activation ends

3) Heaps

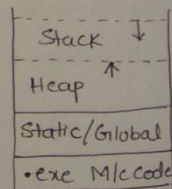
⇒ Allocation and deallocation can be in any order

⇒ Disadv: Heap management is overhead

SUMMARY

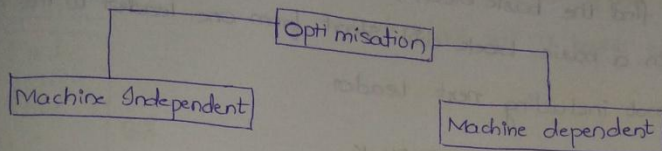
Activations can have

- 1) permanent lifetime in case of static allocation
- 2) Nested lifetime in case of stack Allocation
- 3) Arbitrary lifetime in case of Heap Allocation.



CODE OPTIMISATION

INTRODUCTION TO CODE OPTIMISATION



- 1) Loop optimizations
 - (a) code motion (com)
Frequency reduction
 - (b) Loop unrolling
 - (c) Loop Jamming
- 2) folding
- constant propagation
- 3) Redundancy Elimination
- 4) Strength Reduction

- 1) Register Allocation
- 2) Use of Addressing modes
- 3) peephole optimization
 - (a) Redundant = load/store
 - (b) Strength Reduction
 - (c) flow of control options
 - (d) use of M/C idioms

LOOP OPTIMISATION AND BASIC BLOCKS

- To apply optimisations, we must first detect loops
- For detecting loops we can use control flow Analysis (CFA) using program Flow Graph (PFG)
- To find pfg, we need to find Basic blocks

A basic block is a sequence of 3-Address statements where controls enters at the beginning and leaves at the end without any jumps or halts.

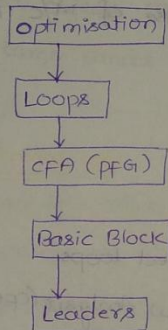
ALGORITHM TO FIND LEADERS

Finding the BASIC BLOCKS

→ In order to find the basic blocks, we need to find the leaders in the program then a basic block will start from one leader to the next leader but not including next leader.

Identifying Leaders in the Basic Block

- 1) I Statement is a leader
- 2) Statement that is target of conditional or unconditional Statement is a Leader $\rightarrow$ if () goto 200 (or) goto 200 $\rightarrow$ leader
- 3) Statement that follows immediately a conditional or unconditional Statement is a Leader.



Example

```
fact(x)
{
    int f = 1;
    for(i = 2; i <= x; i++)
        f = f * i;
    return f;
}
```

Now, the

*1) $f = 1$

2) $i = 2$

*3) if $(1 > x)$

$$*4) t_1 = f$$

5) $t_2 = 1$

7) $\lambda = t_2$;

8) gotu (3)

*9) Gioto co

TYPES

Frequenc

Moving

frequency

frequency

code m

Ex: whi

3

A
j

}

 $t =$

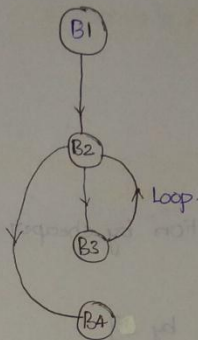
whi

A

Now, the 3-address code for the above problem will be

*1) $f = 1$ B1
 2) $i = 2$
 *3) If $(i > x)$ goto 9 B2
 *4) $t_1 = f * i$;
 5) $t_2 = i + 1$;
 6) $i = t_2$;
 7) goto (3) B3
 *8) Goto calling program B4

⇒ Increase you have 4 leaders we will get
 4 Basic Blocks ⇒ If you have 'n' basic
 Leaders will get 'n' basic blocks



TYPES OF LOOP OPTIMIZATION

Frequency Reduction

Moving the code from high frequency region to low frequency region is called code motion.

Ex: while $(i < 5000)$
 {
 $A = \sin(x) / \cos(x) * i$;
 $i++$;
 }
 ↓
 $t = \sin x / \cos x$
 while $(i < 5000)$
 $A = t * i$;
 $i++$;
 }

Loop unrolling

while $(i < 10)$
 {
 $x[i] = 0$
 $i++$
 }
 ↓
 while $(i < 10)$
 {
 $x[i] = 0$;
 $i++$;
 $x[i] = 0$;
 $i++$;
 }

Loop jamming

combines the bodies of two loops

for $(i = 0; i < 10; i++)$
 for $(j = 0; j < 10; j++)$
 $x[i, j] = 0$;
 for $(i = 0; i < 10; i++)$
 $x[i, i] = 0$
 ↓
 for $(i = 0; i < 10; i++)$
 for $(j = 0; j < 10; j++)$
 {
 $x[i, j] = 0$;
 $x[i, j] = 0$;
 }
 }

MACHINE INDEPENDENT OPTIMISATION

Folding:

Replacing an expression that can be computed at compile time by its value

Ex: $2+3+C+B = 5+C+B$

Redundancy Elimination (DAG)

$$A = B+C$$

$$D = 2+B+3+C$$

$$D = 2+3+A$$

Strength Reduction

Replacing a costly operation by cheaper one

Ex: $B = A * 2$

$$B = A \ll 1 \text{ [Left shift by 1]}$$

Algebraic Simplification

$$\left. \begin{array}{l} A = A + 0 \\ x = x * 1 \end{array} \right\} \begin{array}{l} \text{eliminate such} \\ \text{statements} \end{array}$$

MACHINE DEPENDENT OPTIMISATION

- 1) Register Allocation $\left\{ \begin{array}{l} \text{Local Allocation} \\ \text{Global Allocation} \end{array} \right.$

- 2) Use of Addressing modes

- 3) peephole optimization

- a) Redundant Load and store elimination

$$x = y + z \left\{ \begin{array}{l} \text{Mov } y, R_0 \\ \text{Add } z, R_0 \\ \text{Mov } R_0, x \end{array} \right.$$

$$a = b + c \quad \text{Mov } b, R_0$$

$$d = a + e \quad \text{Add } c, R_0$$

Mov	R ₀ , a
Mov	a, R ₀

X

$$\begin{array}{l} \text{Add } e, R_0 \\ \text{Mov } R_0, d \end{array}$$

(b) Flow control optimisation

Avoid jumps
on jumps

L1: Jump ^{L4}(L2)

L2: Jump L3

L3: Jump L4

Eliminate dead
code

#define x 0

if (x)

{

↓ dead code X

}

d) Use of M/c idioms

$i = i + 1$

mov	R0, i
add	R0, 1
mov	i, R0

 } increment 'i' [inc i]

- ⇒ Handle of the string is a substring that matches with RHS of production
- ⇒ RR conflicts occur in LALR(1) parser when merging of the states
- ⇒ SR conflicts doesnot occur in LALR(1) parser
- ⇒ If the attribute can be evaluated in Depth-first-order then the attribute is L-attributed.